

Refeminizing Creative Computing through a Programming Language for Quilts

Laura Pellowski
Worcester Polytechnic Institute
Worcester, Massachusetts, USA
lpellowski@wpi.edu

Abigail J Haller
Worcester Polytechnic Institute
Worcester, Massachusetts, USA
ajhaller@wpi.edu

Rose Strobel
Worcester Polytechnic Institute
Worcester, Massachusetts, USA
rmstrobel@wpi.edu

Edward Benjamin Tyler
Worcester Polytechnic Institute
Worcester, Massachusetts, USA
ebtyler@wpi.edu

Sakire Arslan Ay
Worcester Polytechnic Institute
Worcester, Massachusetts, USA
sarslanay@wpi.edu

Rose Bohrer
Worcester Polytechnic Institute
Worcester, Massachusetts, USA
rbohrer@wpi.edu



Figure 1: Quilt Designer Home Page

Abstract

We present QuiltDesigner, a programming language for designing quilt patterns packaged in a web interface with a cozy aesthetic. Two user studies with quantitative and qualitative surveys are performed to characterize how QuiltDesigner can support the creativity needs of its programmers. We argue for refeminization of creative computing, i.e., to supplant existing narratives of its femininity with one where language creation and design creation have equal feminine status.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; *Empirical studies in HCI*; • **Software and its engineering** → **Domain specific languages**; • **Applied computing** → **Arts and humanities**.

Keywords

Creative Computing, Quilt Design, Cooties in Computer Science

ACM Reference Format:

Laura Pellowski, Abigail J Haller, Rose Strobel, Edward Benjamin Tyler, Sakire Arslan Ay, and Rose Bohrer. 2025. Refeminizing Creative Computing through a Programming Language for Quilts. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*, April 26–May 01, 2025, Yokohama, Japan. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3706599.3716242>

1 Introduction

Creative coding has received significant research attention [4, 5, 24, 25, 36, 37, 42, 46], including attention from the CHI community [36, 42]. This alt.chi submission shares this advocacy for creative computing, yet provokes toward more radical narratives of what creative computing can be.

The standard focus of creative coding is the development of software that outputs a creative artifact, such as an image, sound, animation, or even tangible object. Creative coding is sometimes pursued for its own sake, for the joy of creation, but is also often integrated into a broader educational context. Pedagogically, creative coding is often rightly motivated by inclusivity concerns, especially gender inclusivity [2, 22]. In a context where masculine-coded interests often predominate classroom examples, creative applications provide a way to show femmes that femme-coded interests have a place in the classroom, promoting a sense of belonging. The authors see ourselves in coalition with works that take this direction, not dismissing them, but calling for new ambitions.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '25, Yokohama, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1395-8/25/04

<https://doi.org/10.1145/3706599.3716242>

This project develops a domain-specific programming language for creating quilt designs, named simply *QuiltDesigner*, implemented as a web application, available at quiltmakermqp.com. The thesis of this paper is not only that programming language design and implementation are *creative*, but that they are *feminine*, and that this matters.

Why does it matter to view programming language creation as a feminine mode of creative expression? It matters to us because the subdivision of computing studies into academic subdisciplines historically functions to recreate patriarchal hierarchies. The authors, like many CHI authors, have seen human-computer interaction dismissed by computer scientists as insufficiently computer-science-like, often with the unstated addition “. . . because it contains too many women.” An analysis of perceived subfield prestige against gender ratio showed a consistent trend that the fewer women researchers in a field, the more concentrated that field’s researchers are at top-ranked institutions [32]. Theoretical subfields of computing were rated as the highest prestige, *including the study of programming languages*.

In feminizing the creation of programming languages, we seek to reclaim computing prestige for femininity, against patriarchy. It is not enough to say that femmes can program, because sexists might counter-argue that our programs, especially creative programs, are mere applications of masculine work in language creation. Nor is it enough to say that femmes can excel in the masculine-coded act of language creation, because this is an admission that we value ourselves for our proximity to masculinity, and will allow masculine norms of discourse to dictate what we say about our work and how. Femme liberation means entitlement to be fully femme and fully computational at the same time, without contradiction, whatever those terms mean to us.

Positionality. We share the authors’ positionalities to contextualize the work. Rose Bohrer and Sakire Arslan Ay are faculty advisors; the remaining authors are their undergraduate advisees. The primary supervisor, Rose Bohrer, is a binary transgender woman. She completed her PhD in programming languages before expanding into HCI, in part due to a more welcoming research community. One student team member, E. Ben Tyler, is a cisgender man. The remaining members of the team are cisgender women.

These team makeup shapes our understanding of femininity. We have experienced its social construction, the reality of misogyny, and the joy of including cisgender men in feminine-coded hobbies. The team makeup reminds us that feminine interests are far from exclusive to feminine genders. On the contrary, the full inclusion of cisgender men in femme-coded work is an anti-patriarchal act. Quoting bell hooks [3]: “in patriarchal culture, males are not allowed simply to be who they are.”

This paper primarily uses the terms femme (feminine) and masc (masculine), not man and woman, because we are interested in activities which our culture has encoded with gender, not gender in senses of identity nor expression. These terms are not meant to imply that all people can be categorized as femme or masc, and we recognize that our choice to express femininity through fabric crafts would not resonate with every reader, femme or not. We have reviewed the HCI Gender Guidelines in choosing this terminology [40].

This project is performed in a university in the northeast USA as an ongoing senior capstone project from August 2024 to March 2025. This both informs the scope of the project and creates limitations of place. Quilting is predominantly a North American craft, and the cultural assumptions underlying the work may not translate to other continents.

The project structure is informed by the peculiarities of our institution’s academic structure. Relevant to CHI, there is no standalone program nor department for HCI, rather the HCI faculty are distributed between the Computer Science (CS) department and the program for Interactive Media and Game Development (IMGD). All team members are in the CS department, but the last author has published in HCI venues and the IMGD program gave the team permission to perform usability studies at their events, which have the explicit goal of providing testing opportunities for their own senior projects.

Paper structure. The introduction and discussion sections are more argumentative in nature while the middle sections are more descriptive, documenting the making of *QuiltDesigner* and initial human subjects studies about it. This mixed structure represents the collaboration between student and faculty team members in preparing the paper; the argumentative sections are primarily written by the faculty team members and the middle sections primarily by the students. This mixture is not a weakness, but intentional. Our argument is that liberation lies in being fully feminine and fully computational at once, thus blending technical reporting with gendered personal experience is a natural methodological decision.

Summary of Motivations. The intrinsic joy of creativity should not be overlooked as a motivation. It is important to remember to have fun, and expression through the arts can be fulfilling to the everyday programmer, and in the research process. The motivation for learning to code is restricted to advancing technical skills and employment reasons too often. While these reasons are productive, gratification through creativity is just as necessary.

Secondly, we wish to reach a larger audience of programmers who may not have experience with quilting. This is partly due to the stereotypical gendering of each hobby. An estimated 98% of quilters in the USA are women [21] while an estimated 80% of professional programmers in the USA are men [12]. Thus, we are able to tackle gender inequalities in quilting by introducing it through a typically male passion. We connect programming, which masculine users already enjoy playing with, to the unfamiliar concept of quilting, so that fulfillment is increased by association.

1.1 Research Question.

The user studies conducted during this project aim to address the following research question: “Which programming language design decisions support programmer creativity in producing quilt patterns?”

This research question is explored both through language-level design decisions, e.g., regarding syntax and naming conventions, as well as user interface-level design decisions.

2 Background and Related Work

We provide brief background information on quilting and a discussion of related work.

2.1 Background: What is a Quilt?

Quilting is a fabric craft traditional to North America. A quilt is an assembly of fabric patches, traditionally used like a blanket, though it is also common to hang quilts like a tapestry. In the quilt-making process, a selection of fabrics and fillings are chosen and cut down into smaller pieces. These pieces are then sewn together, using a needle and thread, either by hand or using a sewing machine. The result is a 2-dimensional sheet of fabric. There is a creative element in quilt making, as the quilter has the liberty of choosing which fabrics to use and how to assemble them together. For the scope of our project, we limit ourselves to the assumption that only rectangle fabric pieces are used. In practice, the quilter can use shapes other than rectangles, with triangles being another popular shape.

Shape is important both to quilting culture and to its computational tools. Prior computational work [13] emphasizes distinct cultures among White quilters and Black quilters, with White quilt culture emphasizing rigid grid-based layouts and Black quilts often exhibiting less rigidity. The authors of prior work then frame the development of flexible digital representation of quilt designs as a matter of racial inclusion within the computational creativity space.

2.2 Related Work

We draw on related works in programming languages and computational crafting as well as feminist writings across disciplines.

2.3 Programming Languages

This project is part of a line of research on programming languages for quilting inspired by Homotopy Type Theory (HoTT) [38, 39], a style of type system which has received substantial recent attention in the programming languages community. We target HoTT for feminist praxis in programming language design because it is considered a prestige topic, is visibly male-dominated, and its trans-feminine researchers in particular have experienced high-profile exclusion [44], albeit with much of the field taking their side. Piece-Work [13] was an initial result in HoTT-based quilting, inspiring ongoing efforts toward simplification [47] on which the present design is based.

Despite long-standing interest in usability among the programming languages community, it is rare for user studies to play a systematic role in the development of academic languages, and rarer yet for user studies and math-heavy formal developments such as type systems to inform each other in a deep way. The PLIERS method is the primary example of this [15], and like our work, makes central use of Wizard-of-Oz studies¹.

2.4 Computational Crafting

This line of work is inspired by the Code Crafters project [23] which explored language-based prototypes before moving toward a

language-free approach for reasons of audience. Barbara Graphics is another key predecessor [49] about programming quilts, but its approach is descended from live coding while ours is algebraic. The Penrose [48] language for mathematical diagrams also influenced our design, particularly of our web UI, serving as a more distant point of comparison. Coahran’s work [14] is not language-based, but provides specialized generative techniques for Bargello quilts. Lamberty explores quilts’ role in constructivist pedagogy [33].

In the knitting space, KnitScape [43], like QuiltDesigner, provides a web interface and repetition support. Albaugh [1, 31], Lin, McCann [35], and collaborators have helped establish mathematical foundations of computational machine knitting, likewise Berstein, Leake [34], and collaborators have established foundations for certain classes of quilts.

Embroidery [28] has been explored as a medium for visualizing data, e.g., input as black-and-white textures. Computational support for crochet [26] has so far emphasized the design stage because the mechanical complexity of crochet fabrication is substantially beyond that of knitting or sewing, thus harder to computerize.

Because many computational crafting tools provide textual input languages, they can also be combined with ostensibly text-focused, formal grammar-driven generative systems like Tracery [16].

2.5 Feminism in Computing

This work contributes to a broader feminist literature within computing, and especially programming languages. Foundational works include Ensmenger’s study of how computing became masculine in the first place [19], showing that its masculine status was not inevitable. The last author’s education research [6–8] foregrounds HCI in the pedagogy of programming languages, in large part because contemporary HCI research provides a language for addressing identity in computing that programming languages academia lacks. A separate project in the formalization of gender pronouns [9] shares this paper’s ideological goal of portraying gender (power) minorities as natural owners of masculine-coded research topics.

Feminist perspectives in programming languages show early signs of spreading. A recent essay by Hermans and Schlesinger [29] again ties the gendering of subfields to patriarchal prestige hierarchies [32]. The essay goes further, applying feminist understandings of the social processes that construct gender to analyze the social processes among researchers which construct disciplinary boundaries.

Our thesis owes a substantial debt to the foundational works of intersectional feminism. Specifically, in her defining works on intersectionality, Crenshaw (e.g., [17]) highlights how people at the intersection of two identities are denied the right to be treated as representative members of either identity, with membership in one being weaponized to “disprove” membership in the other. Crenshaw’s object of study is far graver than ours in its impacts, analyzing how the US criminal justice system denies Black women the protections claimed by anti-discrimination laws. Our goal is not to claim similar gravity, only acknowledge an intellectual debt. Brooke has argued [11] the importance of holding our multiple identities at once within creative computing spaces specifically. Brooke showed how even creative spaces such as hackathons can

¹Though PLIERS did not directly inspire our decision, it is a twist of fate that one of the authors first learned of such studies from one of the PLIERS authors.

enforce sexist cultures of computing, and that programming language discourse is one means by which they do so. It is thus no coincidence that our paper emphasizes the simultaneous identities of femme, programming language designer, and creative.

3 QuiltDesigner Language and Interface

We present QuiltDesigner by giving a primer on language syntax, describing its implementation, and walking through interface usage. Readers preferring a visual introduction are encouraged to skip to Section 3.3.

3.1 Language Primer

This section introduces the syntax of the QuiltDesigner programming language by example. The fundamental building block is a Patch, which is simply a rectangle shape with a width, height, and color. Colors are referred to by name; for reasons of scope, definitions of each color name are hard-coded in the prototype. The user can create and combine Patches to create a larger Design.

```
rect(width, height, color);
```

There are several built-in ways to combine Patches together. The first two are `hor()` and `vert()` which place the specified rectangles next to each other horizontally, or on top of each other vertically, respectively. To successfully place the Patches, the heights need to be the same if using `hor()`, and the widths need to be the same for `vert()`.

```
hor(Patch, Patch, ...);
vert(Patch, Patch, ...);
```

The user can also repeat a Patch a specified number of times using `repX()` or `repY`, respectively for horizontal and vertical repetition. Though repetition is much less powerful than `while` or `for` loops, this tradeoff was essential, because it ensures the dimensions of a quilt can be more readily analyzed by inspecting the code.

```
rep(Number, Patch);
```

If the user would like to place Patches directly on top of one another, they can do so using `over()`. We have 5 possible Position options to choose from: TL, TR, BL, BR, C for top left, top right, bottom left, bottom right, and center, respectively.

```
over(Position, Patch, Patch, ...);
```

To rotate a Patch, `rot()` is used. The Angle is limited to the following values: 0, 90, 180, 270.

```
rot(Angle, Patch);
```

To make the programming experience a bit easier, we also have implemented variables and functions. The user has to specify the type of the variable when declaring it. A variable definition for a Patch would look like the following:

```
rect patch = rect(width, height, color);
```

Likewise, for a number variable definition, the following syntax should be used:

```
int x = 5;
```

A function definition can implement rectangles and/or numbers as parameters.

```
define functionName (rect x, rect y) { return
    expression }
```

3.2 Language Implementation

We describe the QuiltDesigner implementation, which consists of a parser (transforming textual code to an appropriate representation), an interpreter (computing which quilt design results from the code), and the web frontend, including a visualizer that renders resulting quilt designs onto the screen.

Parser. Related work proposes a syntax [47]. Our parser implements a slightly modified syntax which, for example, allows writing colors by their names instead of by numeric values. The parser is implemented in JavaScript using the tool `Peggy.js` [30], which required adapting the proposed syntax into a style known as a Parsing Expression Grammar (PEG) [20]. The parser outputs a tree-shaped data structure called an Abstract Syntax Tree (AST).

3.2.1 Interpreter. The interpreter takes in the AST and computes its result, a Design consisting of Patches. It outputs JavaScript objects that store all of the information needed to draw the quilt to the screen, such as coordinates, sizes, and colors. A Design object contains a list of Patch objects, and an overall width and height.

3.2.2 Web Interface. The frontend of QuiltDesigner is structured as a web application for ease of access. Throughout implementation, our design process focused on visual aesthetics.

We developed a mood board to characterize our aesthetic goals, containing colors, fonts, and other specific style elements that inspired us. Discussing the mood board in design meetings, we settled on “cozy” as the aesthetic core. We focused on inviting visuals, featuring pastel colors and a digitally painted homepage. Our initial color palette emphasized pastel blues and cream colored whites. Iterating on the theme of coziness, we transitioned to a fall themed color set: cream colored white with a dark orange accent for the navbar and button elements. The cozy aesthetic also guided font choice (Roboto and Calibri) and element styling choices like rounding, to convey safety and friendliness. The video game *Fields of Mistria* [41] inspired our home screen visuals.

The mood board also included inspiration for specific UI components. Our example gallery component is inspired by Penrose [48], our tutorial page by CodeMirror [27], and our canvas-based visualizer by Barbara.

We developed UI wireframes in order to efficiently iterate the visual design, resolving issues such as button layout at an early stage.

We then proceeded to full implementation of the web frontend in HTML, JavaScript, and CSS, using React.js and the CodeMirror [27] text-editor component.

3.3 Walkthrough

This section walks the reader through interaction with the web interface. The drawn homepage is depicted in Figure 1, with an animated rain background.

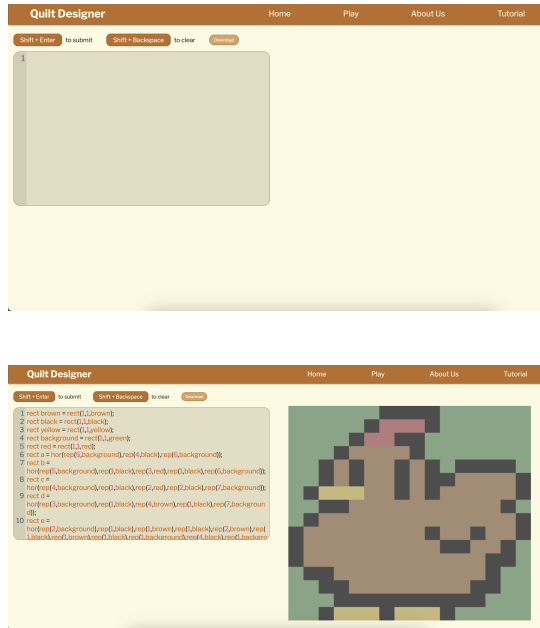


Figure 2: CodeMirror Editor and Canvas. Top: upon opening; Bottom: after designing a chicken

The Start button loads the main editor interface (Figure 2), with a text editor on the left and a blank canvas for visualization on the right. Instructions such as keyboard shortcuts are provided on-screen (Visualize: Shift+Enter, Erase All: Shift+Backspace). The Download button provides the visualization in .png format.

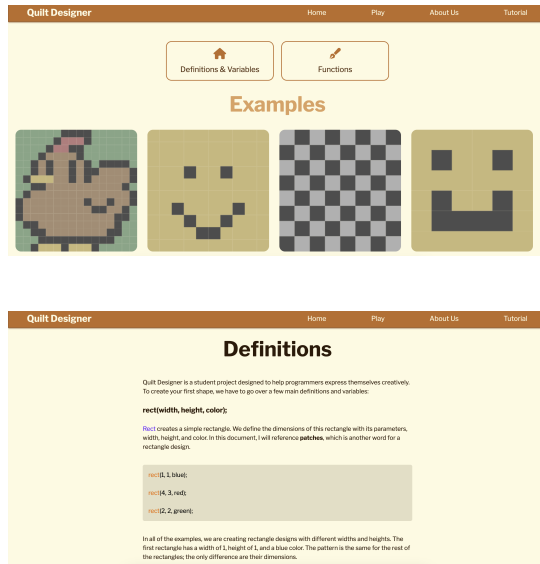


Figure 3: Top: Examples Gallery, Bottom: Documentation

The Tutorial link leads to a curated example gallery, from which further links (e.g. Definitions & Variables) lead to documentation

from the team; both are depicted in Figure 3). Selecting an example copies it into the editor, allowing experimentation without fear of permanent changes.

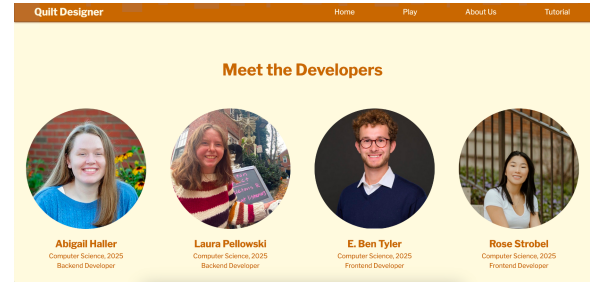


Figure 4: About Us Page

An About Us page (Figure 4) is provided to document the project goals and each student's project roles.

4 User Studies

This section describes two user studies conducted on the authors' campus. These studies are approved by WPI's Institutional Review Board (IRB), record number IRB-25-0054. Our IRB routinely reviews applications for minimal-risk human subjects research in the context of senior projects, and students are routinely provided training in the preparation of consent forms and data management. Our games program (IMGD) routinely organizes events for student projects to gather feedback through human subjects research. Our two rounds of studies occurred at two such events named *Protofest* (October) and *Alphafest* (November), named to reflect the expect stages of progress at each event. The environment at both events was similar: because many projects are soliciting feedback, the length of engagement with each project is limited, thus the studies are designed to assume 10 to 20 minutes of engagement.

Pursuant to our main research question, both studies aim to learn which design decisions support programmer creativity, how, and how well. Each study consists of an open-ended creative design session followed by a survey.

4.0.1 Study 1: Wizard-of-Oz Study at Protofest. This study was conducted before significant technical implementation occurred. Formative in nature, its goal was to provide design iteration and prioritize implementation goals.

At this early stage, we performed a *Wizard-of-Oz* study [18]: lacking a prototype, the experimenter manually simulated programming language behavior. Participants interacted with the language by typing commands into Google Docs, which a team member manually interpreted and visualized using Pixilart.com. Wireframes were also presented so that visual UI design could be assessed.

Participants were encouraged to explore the language and then answer these questions:

- How do you see this program being used?
- What features would you like to see added to this program?
- What do you see yourself making with this program?
- What aspects of the programming language did you find enjoyable?

- What aspects of the programming language did you find hard to use?
- The open-ended nature of these questions supported brainstorming and iterative design, ensuring that user feedback directly informed the language's future development.

4.0.2 Study 2: Full Prototype at Alphafest. This November study replaces the Wizard-of-Oz study with a fully functional prototype. Data were collected through Google Forms for efficiency and consistency. The feedback is a mixture of formative and summative, including high-level experiential feedback as well as specific areas targeted for improvement such as error message quality:

- How would you describe the overall experience of using the website to create designs?
- Which commands or features of the language did you find most useful? Why?
- Were there any commands or syntax you struggled to understand or use effectively? Please explain.
- Did the language provide enough flexibility for creating the designs you envisioned? If not, what would improve this? What features might you add?
- Were the visualizations accurate and clear representations of the code you entered? (Yes/No – If No, explain.)
- What specific aspects of the UI (e.g., specific UI elements, layout, visual design) worked well? Why?
- Were there any UI elements that felt cumbersome or confusing? Please explain.
- How well did the error messages help you debug your code? (1 = Not Helpful, 5 = Very Helpful – If less than 4, explain why.)
- Were there any moments where you felt stuck or unsure of what to do next? If so, what moment? What caused it?
- What was your favorite design during this session?
- If you were to use the finished version of this program, what kinds of projects or designs would you create?
- Any additional comments, suggestions, or feedback you'd like to share?

5 Data Analysis

Throughout the project, informal qualitative analysis of survey results has driven team decision making. In preparing this article, we elaborated the analysis into a more structured style. The research team has experience in reflexive thematic analysis (RTA) [10], but the analysis presented here is not formally RTA because we lack an explicit coding step. Instead, the domain summaries and themes discussed here emerged through a process of identifying key quotes in the survey data and discussing these data among the team. We also arrive at our themes in a more specific way: for each domain identified in the data, we compare data from the first and second study to identify trends over time.

Each survey received responses from 13 participants. In general, they were not the same 13 participants, as attendees explored the events freely.

5.1 Study 1

In reading and discussing results of the Protofest survey, three main topics emerged. In RTA terminology, we present these three topics

as *domain summaries*, not *themes*, meaning we seek only to identify which domains were of interest to respondents, not draw shared meaning across their responses.

Accessibility of Syntax. This domain captures participants' challenges with understanding the programming language and using its commands. Feedback consistently highlighted difficulties with syntax. Because the study was hosted at an IMGD event, many non-coders were present, and they struggled particularly. One participant explicitly noted, "*Syntax can be frustrating for non-coders.*" Participants emphasized the importance of tutorials and examples, with one suggesting that "*having example code would make the tool easier to use, particularly for beginners.*"

UI Design and Usability. This domain captures the layout, aesthetics, and usability of the proposed UI. Some participants provided concrete revisions to the wireframes. One participant suggested, "*Spacing between 'Quilt,' 'Designer,' and 'Play' should be equal.*" Another noted that "*the font could be cozier to match the overall feel.*"

Feature Requests. This domain captures participants' numerous feature requests, primarily toward increasing creative expressiveness. Comparing the complexity and frequency of requests helped the team prioritize future development. Requests ranged from common layouts (checkerboards, 3 by 3 patch grids) and new shapes (e.g., right triangles) to features for vertical and horizontal repetition.

Feedback within each domain drove future development, respectively: we added an example database to mitigate syntax challenges, we revised layout and committed to the cozy aesthetic based on UI feedback, and we focused on adding repetition features because they were both technically feasible and highly requested.

5.2 Study 2

We analyzed the data from Study 2 by revisiting each domain identified in Study 1, searching for responses that indicate the resolution of prior problems and the existence of future room for improvement.

Accessibility of Syntax. After adding an example gallery, Study 2 responses showed more positive sentiment toward the syntax compared to Study 1. Several participants praised the language's core commands, with one noting, "*The rep and rect commands were particularly intuitive.*" However, confusion remained about how to use specific commands to achieve more complex tasks. For instance, one participant struggled to understand the over command when creating a smiley face design. Error messages were highlighted as an area for further work; one participant described them as "*vague*," because a spelling mistake failed to generate a helpful error.

UI Design and Usability. The frontend received the most positive feedback out of all parts of the survey. Participants praised the cozy aesthetic of the (digitally) hand-drawn visuals and viewed the revised UI design as clean. None of the participants gave negative feedback about visual design in Study 2.

Feature Requests. Feature requests decreased from Study 1 to Study 2. This is expected, because Study 2 evaluated a concrete

prototype while Study 1 did not. The requested features did indicate one important design limitation: one participant requested a diagonal line feature, but this may conflict with our grid-based approach. Other suggestions, like a design randomization button, are consistent with our grid-based approach.

5.3 Analysis Across Studies

We compared our observations from Study 1 and Study 2 to draw trends in the data. In RTA terminology, we arrived at the following themes, one for each domain, by comparing the data within each domain across Study 1 vs. Study 2. These themes serve as our answer to the primary research question of the paper.

- **Tutorials and Examples Improve Accessibility:** The addition of an example database at Alphafest significantly reduced confusion and increased engagement times.
- **UI Refinements Enable Aesthetic Appreciation:** Study 1 helped clarify our aesthetic goals. Smaller revisions, such as layout improvements, made it easier to appreciate those aesthetics.
- **Feature Requests Highlight Creative Aspirations:** Participants in both studies thought of diverse applications for the language, from heraldry to pixel art. Their requests for new shapes, patterns, and randomness reflect the language's potential as a creative outlet.

The third theme reinforces our thesis that language development is as much an act of creativity (and a feminine one, at that) as the design of quilts. A language designer, faced with many feature requests, uses their intuition and creativity to guide language design, participating in a process of co-creativity with the programmers who use the language.

6 Discussion

We reflect on what we mean when we call the present work feminine, then look forward to the project's implications for disciplinary dynamics between CS and HCI. We discuss known limitations of the work and directions for future work.

6.1 Where was the Femininity?

We argue that this work is feminine, yet the writing of the technical sections might strike readers as lacking visible femininity. Why is this?

We must first decide what we mean when we mean femininity. If we are femmes and we develop programming languages, then technical writing is feminine writing, and we merely need expand our definitions.

However, this definitional concern raises a more subtle point: masculine writing norms dictate which experiences are written about in technical academic contexts. If we define femininity in experiential terms, i.e., a feminine moment is any moment where a team member felt *I am feminine right now*, then these moments were rarely the most technical ones. When we say we are denied simultaneous femininity and technicality, we mean this, that we are told technical writing has no place for celebrating our feelings. When femmes perform masc-coded work and document it according to masculine norms, feminine experiences may not be visible *because we succeeded in replicating a masculine standard*. To

write this article in a feminine style means to cherish moments in the project that would not be cherished by a masculine norm of evaluation for research articles.

To this end, the last author recalls feminine moments most important to her. After completing a project milestone, the student team got together to sew a quilt as a group, then brought sewing supplies to our next advisor meeting so the faculty advisors could participate in making the quilt. This meant a lot to me because I worry about my students overworking and never taking the time to enjoy college. When I do this worrying, I feel like a mother, I feel feminine. To see us all smiling while working with a needle and thread, felt girly. When I made a mistake sewing and felt embarrassed, the way in which I expressed my frustration felt like a micro-affirmation of femininity. It was likewise affirming to watch the sole male team member thoughtfully navigate a femme-dominated space, his very carefulness reaffirming that I belong to the group which he is being careful not to speak over.

None of these moments are deliverables. They are not p-values and standard deviations, not code nor machines, and certainly not theorems we prove by induction. They are the memories of belonging, and as long as research insists only on publicizing the statistics, artifacts, and mathematical laws, it will fail to characterize the experience of belonging in adequate qualitative detail. When we as scholars study *how* to build belonging in technical spaces, why do we feel afraid to report the moments where we *as humans* felt the most ourselves?

6.2 Implications for HCI-CS Relations

Gender means something different in HCI than in CS, and it is important context that the researchers perform HCI work in a CS department. For example, readers have by now likely begun to check whether our lens on gender adequately includes non-binary people, as we intend to. In contrast, during the senior author's PhD in programming language theory, she once tried to assign they/them pronouns to an anthropomorphized mathematical object. This resulted in a six-month argument which she lost for being "too political."

We hope to spark conversation between attendees homed in CS departments vs. those homed in HCI departments and programs, because our experiences of gender likely differ. In an HCI setting, it is common to look around and see many visible examples of femininity, to even have regular access to femme-dominated spaces; for HCI scholars in CS departments, the mere mention of femininity can lead to accusations that one's work is insufficiently CS.

Because this project is a senior capstone, we also hope to spark conversation about mentorship, especially strategies for adapting mentorship to each institution. The senior author is her institution's only full-time faculty member with specialty in programming language foundations and the institution has a relatively modest group of HCI faculty. Our HCI course offerings are minimal, with only an introduction-to-HCI course and a data visualization course offered regularly. In this context, our human subjects studies mean more than they mean within an HCI degree. These represent a way to take bread-and-butter HCI research methods and sneak them into a CS-dominated curriculum. Through project work that blends disciplinary boundaries, faculty create opportunity for human-minded

students to find each other, discover what HCI can be, and consider further HCI study seriously for the first time. Indeed, the preparation of this paper was motivated in large part by the hope of helping student authors attend their first CHI and become more familiar with the academic career path.

6.3 Limitations

We discuss three kinds of limitations: limitations of scope, limitations of methodology, and limitations of positionality. The time constraints of a senior capstone project required us to manage and limit technical scope. We kept a running list of desired features ranked by priority, using feedback from user studies to prioritize. Our surveys included quantitative questions which, for our sample size, do not reach the threshold of statistical significance. This decision was made to manage the time spent answering and prevent survey fatigue. It is important that the results of such questions be understood as an imperfect means of formative feedback that shaped design iteration, not a reproducible summative evaluation. Our respondents are drawn from our local campus community, especially from students in the games program. This program has fewer students with programming experience compared to the research team, which contextualizes usability challenges in the initial study. The gender, racial, and socioeconomic distributions of this community are also not representative of the USA nor the world, and are not expected to generalize. Likewise, the authors' positionality as USA-based researchers limits our cultural perspective to the USA context. For example, when the last author presented a related project [13] to a Japanese audience, many of the questions were about defining quilts, because quilting is a culture-specific craft. This experience informed our efforts to provide background information about quilting in this article.

6.4 Future Work

Short-term future work focuses on functional improvements. We hope to include additional shapes like triangles and stars, and additional operations like mirroring. Potential additions to the web interface include a gallery of user projects to encourage community involvement, and a save function.

Long-term, *QuiltDesigner* is one part of a larger agenda. We plan to refine and publish the proposed theoretical foundations of the language. Outreach activities are an important aspect of the project, with the interactive demonstration at `alt.chi` being one such outreach activity.

References

- [1] Lea Albaugh, James McCann, Scott E. Hudson, and Lining Yao. 2021. Engineering Multifunctional Spacer Fabrics Through Machine Knitting. In *CHI '21: CHI Conference on Human Factors in Computing Systems, Virtual Event / Yokohama, Japan, May 8-13, 2021*, Yoshifumi Kitamura, Aaron Quigley, Katherine Isbister, Takeo Igarashi, Pernille Bjørn, and Steven Mark Drucker (Eds.). ACM, 498:1–498:12. doi:10.1145/3411764.3445564
- [2] William H Bares, Bill Manaris, Renée McCauley, and Christine Moore. 2019. Achieving gender balance through creative expression. In *Proceedings of the 50th ACM technical symposium on computer science education*. 293–299.
- [3] bell hooks. 2004. *The Will to Change: Men, Masculinity, and Love*. Washington Square Press.
- [4] Ilias Bergstrom and R Beau Lotto. 2015. Code Bending: A new creative coding practice. *Leonardo* 48, 1 (2015), 25–31.
- [5] Marina Umaschi Bers and Amanda Sullivan. 2020. *ScratchJr coding cards: creative coding activities*. No Starch Press.
- [6] Rose Bohrer. 2023. Centering Humans in the Programming Languages Classroom: Building a Text for the Next Generation. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on SPLASH-E, SPLASH-E 2023, Cascais, Portugal, 25 October 2023*, Molly Q. Feldman and Michael Hilton (Eds.). ACM, 26–37. doi:10.1145/3622780.3623646
- [7] Rose Bohrer. 2023. *Human-Centered Programming Languages*. Self-published. bookish.press
- [8] Rose Bohrer. 2024. Programming Language Case Studies Can Be Deep. In *Proceedings of the Thirteenth Workshop on Trends in Functional Programming in Education, TFPiE 2024, South Orange, New Jersey, USA, 9th January 2024 (EPTCS, Vol. 405)*, Stephen Chang (Ed.). 56–79. doi:10.4204/EPTCS.405.4
- [9] Rose Bohrer and Ashe Neth. 2024. Pronoun Logic. *CoRR* abs/2409.18978 (2024). doi:10.48550/ARXIV.2409.18978 arXiv:2409.18978 Presented at Queer in AI@NAACL 2024.
- [10] Virginia Braun and Victoria Clarke. 2019. Reflecting on reflexive thematic analysis. *Qualitative research in sport, exercise and health* 11, 4 (2019), 589–597.
- [11] Sian Brooke. 2020. *Breaking gender code: Visibility, power, and gender in creative coding cultures*. Ph. D. Dissertation. University of Oxford.
- [12] U.S. Census Bureau. 2023. <https://datausa.io/profile/soc/computer-programmers>
- [13] Charlotte Clark and Rose Bohrer. 2023. Homotopy Type Theory for Sewn Quilts. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Functional Art, Music, Modelling, and Design (Seattle, WA, USA) (FARM 2023)*. Association for Computing Machinery, New York, NY, USA, 32–43. doi:10.1145/3609023.3609803
- [14] Marge M. Coahran and Eugene Fiume. 2005. Sketch-Based Design for Bargello Quilts. In *2nd Eurographics Workshop on Sketch-Based Interfaces and Modeling, SBIM 2005, Dublin, Ireland, August 28-29, 2005, Proceedings*, Joaquim Armando Pires Jorge and Takeo Igarashi (Eds.). Eurographics Association, 165–174. doi:10.2312/SBM/SBM05/165-174
- [15] Michael J. Coblenz, Gauri Kambhatla, Paulette Koronkevich, Jenna L. Wise, Celeste Barnaby, Joshua Sunshine, Jonathan Aldrich, and Brad A. Myers. 2021. PLIERS: A Process that Integrates User-Centered Methods into Programming Language Design. *ACM Trans. Comput. Hum. Interact.* 28, 4 (2021), 28:1–28:53. doi:10.1145/3452379
- [16] Kate Compton and Michael Mateas. 2015. Tracery: An Author-Focused Generative Text Tool. In *Proceedings of the 10th International Conference on the Foundations of Digital Games, FDG 2015, Pacific Grove, CA, USA, June 22-25, 2015*, José P. Zagal, Esther MacCallum-Stewart, and Julian Togelius (Eds.). Society for the Advancement of the Science of Digital Games. http://www.fdg2015.org/papers/fdg2015_extended_abstract_18.pdf
- [17] Kimberlé Crenshaw. 2013. Demarginalizing the intersection of race and sex: A black feminist critique of antidiscrimination doctrine, feminist theory and antiracist politics. In *Feminist legal theories*. Routledge, 23–51.
- [18] Nils Dahlbäck, Arne Jönsson, and Lars Ahrenberg. 1993. Wizard of Oz studies: why and how. In *Proceedings of the 1st International Workshop on Intelligent User Interfaces, IUI 1993, Orlando, Florida, USA, January 4-7, 1993*, Wayne D. Gray, William E. Hefley, and Dianne Murray (Eds.). ACM, 193–200. doi:10.1145/169891.169968
- [19] Nathan L Ensmenger. 2012. *The computer boys take over: Computers, programmers, and the politics of technical expertise*.
- [20] Bryan Ford. 2004. Parsing expression grammars: a recognition-based syntactic foundation. In *Proceedings of the 31st ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 111–122.
- [21] Abby Glassenberg. 2020. Quilting trends survey results 2020. <https://craftindustryalliance.org/quilting-trends-survey-results-2020/>
- [22] Hanneke Goud. 2024. *Using graphic design to attract female high school students to the field of creative coding*. B.S. thesis. University of Twente.
- [23] Jordan Graves, Karen Royer, Gillian Smith, and Anne Sullivan. 2021. Procedural Patchwork: Community-Focused Generative Design for Quilting. In *Proceedings of the 13th Conference on Creativity and Cognition (Virtual Event, Italy) (Creativity and Cognition)*. Association for Computing Machinery, New York, NY, USA, Article 56, 3 pages. doi:10.1145/3450741.3466635
- [24] Ira Greenberg. 2007. *Processing: creative coding and computational art*. Springer.
- [25] Ira Greenberg, Deepak Kumar, and Dianna Xu. 2012. Creative coding and visual portfolios for CS1. In *Proceedings of the 43rd ACM technical symposium on Computer Science Education*. 247–252.
- [26] Runbo Guo, Jenny Lin, Vidya Narayanan, and James McCann. 2020. Representing Crochet with Stitch Meshes. In *SCF '20: Proceedings of the 5th Annual ACM Symposium on Computational Fabrication, Virtual Event, USA, November 5-6, 2020*, Emily Whiting, John Hart, Cynthia Sung, Nadya Peek, Masoud Akbarzadeh, Daniel Aukes, Adriana Schulz, Hayden Taylor, and Jeeun Kim (Eds.). ACM, 4:1–4:8. doi:10.1145/3424630.3425409
- [27] Marijn Haverbeke et al. 2024. *CodeMirror*. <https://codemirror.net/5/index.html>
- [28] Tingying He, Petra Isenberg, and Tobias Isenberg. 2023. Data Embroidery with Black-and-White Textures. *CoRR* abs/2309.02864 (2023). doi:10.48550/ARXIV.2309.02864 arXiv:2309.02864
- [29] Felienne Hermans and Ari Schlesinger. 2024. A Case for Feminism in Programming Language Design. In *Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and*

- Software. 205–222.
- [30] Joe Hildebrand. 2024. *Peggy.js*. <https://peggyjs.org/index.html>
 - [31] Megan Hofmann, Lea Albaugh, Tongyan Wang, Jennifer Mankoff, and Scott E. Hudson. 2023. KnitScript: A Domain-Specific Scripting Language for Advanced Machine Knitting. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 2023, San Francisco, CA, USA, 29 October 2023– 1 November 2023*, Sean Follmer, Jeff Han, Jürgen Steimle, and Nathalie Henry Riche (Eds.). ACM, 21:1–21:21. doi:10.1145/3586183.3606789
 - [32] Nicholas LaBerge, K Hunter Wapman, Allison C Morgan, Sam Zhang, Daniel B Larremore, and Aaron Clauset. 2022. Subfield prestige and gender inequality in computing. *arXiv preprint arXiv:2201.00254* (2022).
 - [33] K. K. Lamberty. 2014. Useful views of student artifacts and activities: Supporting student use of an open-ended exploratory construction tool in the classroom. In *2014 International Conference on Collaboration Technologies and Systems, CTS 2014, Minneapolis, MN, USA, May 19–23, 2014*. IEEE, 361–362. doi:10.1109/CTS.2014.6867589
 - [34] Mackenzie Leake, Gilbert Bernstein, Abe Davis, and Maneesh Agrawala. 2021. A mathematical foundation for foundation paper pieceable quilts. *ACM Trans. Graph.* 40, 4 (2021), 65:1–65:14. doi:10.1145/3450626.3459853
 - [35] Jenny Lin, Vidya Narayanan, Yuka Ikarashi, Jonathan Ragan-Kelley, Gilbert Bernstein, and James McCann. 2023. Semantics and Scheduling for Machine Knitting Compilers. *ACM Trans. Graph.* 42, 4 (2023), 143:1–143:26. doi:10.1145/3592449
 - [36] Andrew M McNutt, Anton Outkine, and Ravi Chugh. 2023. A Study of Editor Features in a Creative Coding Classroom. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–15.
 - [37] Kylie Pepler and Yasmin Kafai. 2005. Creative coding: Programming for personal expression. Retrieved August 30, 2008 (2005), 314.
 - [38] The Univalent Foundations Program. 2013. Homotopy type theory: Univalent foundations of mathematics. *arXiv preprint arXiv:1308.0729* (2013).
 - [39] Egbert Rijke. 2022. Introduction to homotopy type theory. *arXiv preprint arXiv:2212.11082* (2022).
 - [40] Morgan Klaus Scheuerman, Katta Spliel, Oliver L. Haimson, Foad Hamidi, and Stacy M. Branham. 2020. HCI Guidelines for Gender Equity and Inclusivity. <https://www.morgan-klaus.com/gender-guidelines.html>
 - [41] NPC Studio. 2024. Fields Of Mistria. video game.
 - [42] Jakob Tholander and Martin Jonsson. 2022. Co-coding with AI in creative programming education. In *The 2022 CHI Conference on Human Factors in Computing Systems (CHI EA'22)*. Association for Computing Machinery (ACM).
 - [43] Hannah Twigg-Smith, Emily Whiting, and Nadya Peek. 2024. KnitScape: Computational Design and Yarn-Level Simulation of Slip and Tuck Colorwork Knitting Patterns. In *Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, Honolulu, HI, USA, May 11–16, 2024*, Florian 'Floyd' Mueller, Penny Kyburz, Julie R. Williamson, Corina Sas, Max L. Wilson, Phoebe O. Toups Dugas, and Irina Shklovski (Eds.). ACM, 860:1–860:20. doi:10.1145/3613904.3642799
 - [44] Username ThePuzzlemaker. 2022. Use inclusive language in §1.11. GitHub issue. <https://github.com/HoTT/book/pull/1101>
 - [45] Bryan Ware. 2024. Make Pixel Art Online. <https://www.pixilart.com/draw>
 - [46] Karen Woo and Garry Falloon. 2022. Problem solved, but how? An exploratory study into students' problem solving processes in creative coding tasks. *Thinking Skills and Creativity* 46 (2022), 101193.
 - [47] Yichi Xu, Laura Pellowski, Karen Royer, and Rose Bohrer. 2024. Cubical Type Theory as a Foundation for Shape Languages. (2024). Unpublished preprint.
 - [48] Katherine Ye, Wode Ni, Max Krieger, Dor Ma'ayan, Jenna Wise, Jonathan Aldrich, Joshua Sunshine, and Keenan Crane. 2020. Penrose: from mathematical notation to beautiful diagrams. *ACM Trans. Graph.* 39, 4 (2020), 144. doi:10.1145/3386569.3392375
 - [49] Kit Zellerbach and Charlie Roberts. 2021. Live Coding Languages For Quilters. <https://www.barbara.graphics/about>

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009